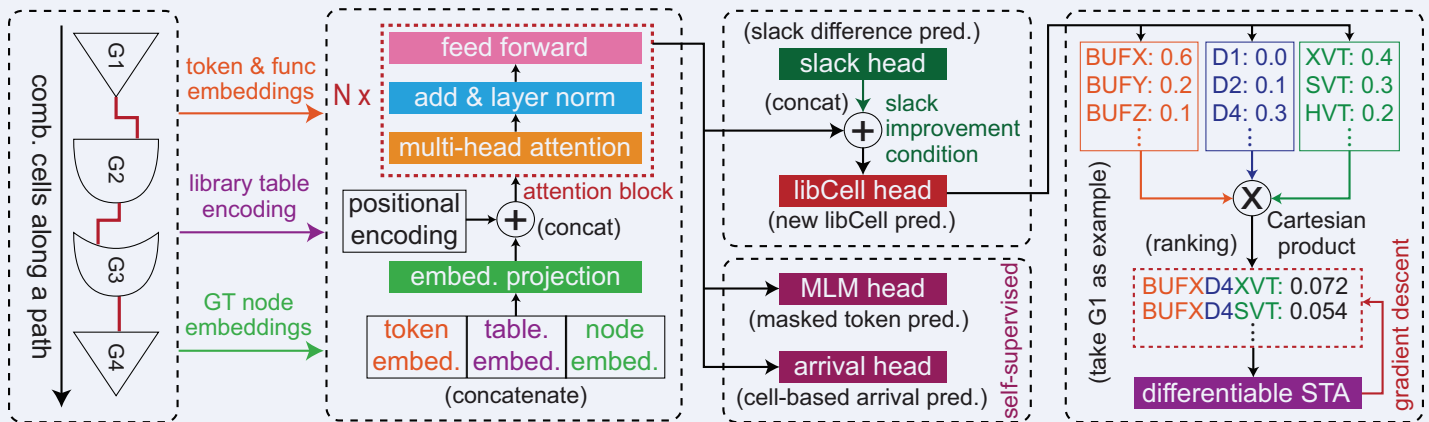


AI-Assisted IC Design With Accelerated Computing and LLM-Driven Optimization

Toward Faster, Smarter Electronic Design Automation



CREDIT: YI-CHEN LU, HAO-HSIANG HSIAO, RONGJIAN LIANG, AND HAOXING REN

Modern physical design (PD) flows face unprecedented scalability challenges as integrated circuits (ICs) reach scales of billions of transistors at advanced nodes. The combinatorial explosion of the design space, coupled with the intricate tradeoffs among power, performance, and area (PPA), increasingly

strains traditional heuristic-based optimization. This has motivated a shift toward artificial intelligence (AI)-assisted design methodologies, which leverage data-driven techniques to efficiently navigate high-dimensional design spaces. At the same time, AI-driven PD critically relies on fast and accurate physical feedback since learning-based methods typically require thousands of iterative evaluations during training or inference. This need has driven substantial advances in GPU-

accelerated PD, which allow scalable physical evaluation within tight design loops. In this article, we review major directions in AI-assisted scalable PD, including GPU-accelerated PD, machine learning (ML)-based modeling, reinforcement learning (RL), differentiable optimization, and large language model (LLM)-assisted design automation. Representative work includes INSTA [1], which facilitates efficient gradient-based timing optimization by introducing a GPU-accelerated differentiable

static timing analysis (STA) engine with near-signoff accuracy. Lib2Vec [2] learns transferable standard-cell representations to reduce reliance on handcrafted features and enable cross-library reuse. LEGO-Size reformulates gate sizing as a language modeling task, achieving a 125× speedup and a 37% reduction in total negative slack (TNS) over commercial tools on unseen 3-nm designs. Extending this paradigm to buffer tree synthesis, BUFFALO [3] demonstrates further timing improvements across varying designs. Together, these approaches demonstrate how AI-assisted automation provides a scalable and generalizable extension to traditional PD flows.

Introduction

The semiconductor industry stands at a critical juncture. As Moore's Law continues to drive transistor scaling, modern systems-on-chip (SoCs) integrate billions of transistors, posing unprecedented challenges for traditional electronic design automation (EDA) methodologies. In particular, PD, which transforms circuit netlists into manufacturable layouts, has become increasingly complex due to an exponentially expanding solution space, aggressive time-to-market constraints, and stringent PPA requirements.

Traditional PD tools rely on carefully crafted heuristics developed through decades of engineering effort. While effective for earlier technology generations, these approaches face fundamental limitations at advanced technology nodes. First, the sheer scale of modern designs—often involving tens of millions of placeable instances, highly complex routing patterns, and billions of timing arcs—renders exhaustive optimization computationally intractable. Second, strong interdependencies among design variables create an NP-hard combinatorial optimization landscape in which heuristic methods frequently converge to suboptimal solutions or become prohibitively expensive to execute. Third, the increasing diver-

sity of technology nodes and design methodologies calls for optimization strategies that can generalize across heterogeneous design contexts to deliver high-quality solutions under tight turnaround constraints.

These challenges have motivated growing interest in AI-assisted PD, which combines data-driven optimization with parallel computation to efficiently navigate large and high-dimensional design spaces at scale. Existing approaches toward modern scalable PD primarily fall into the following categories.

GPU-Accelerated PD: Effective AI-driven PD critically depends on rapid and accurate physical feedback to support iterative optimization loops. This requirement has driven growing interest in GPU-accelerated PD. DREAMPlace [4] pioneered GPU-accelerated analytical placement by reformulating wirelength and density optimization as differentiable operations on the GPU, leveraging deep learning tool kit infrastructure (e.g., PyTorch) to map the electrostatic-based placement problem onto massively parallel GPU architectures, achieving orders-of-magnitude speedups over CPU-based placers. Subsequent works, including DREAMPlace 4.0 [5], extended this framework to timing-driven placement by introducing a momentum-based net weighting scheme that dynamically adjusts net weights during gradient-based optimization based on path criticality, enabling timing awareness without requiring explicit differentiable timing computation. Guo et al. [6] introduced GPU-accelerated STA by parallelizing leveled timing propagation on GPUs, demonstrating significant speedups for STA and motivating a series of follow-up works on GPU-based timing computation. GAMER [7] demonstrates that GPU-accelerated maze routing can substantially accelerate routing search. In addition, GPUs have been leveraged to accelerate other traditionally expensive EDA tasks. For gate-level logic simulation, SimPart [8] introduces a replication-aided partitioning strat-

egy that minimizes inter-thread synchronization by replicating a small fraction of overlapping logic across partitions, achieving significant simulation speedups with negligible area overhead; other GPU-accelerated simulation efforts include GEM [9] and GATSPI [10]. For gate sizing, Chung et al. [11] demonstrate that the inherent parallelism across independent timing paths can be effectively exploited on GPUs to accelerate the iterative delay-evaluation and cell-selection steps in gate sizing, substantially reducing runtime compared to CPU-based implementations.

ML for EDA: In recent years, ML applications have grown rapidly in EDA [12]. Among these, graph neural networks (GNNs) have emerged as an effective paradigm for learning over circuit structures [13], [14], [15]. Beyond predictive modeling, prior work has demonstrated the effectiveness of GNN-based models for PD quality estimation, including power prediction [16], [17] and doomed-run prediction [18]. ML has also been applied to placement optimization, including generative adversarial network (GAN)-based placement quality enhancement [19], [20], PPA-directed clustering [21], and affinity-aware optimization [14]. For 3D ICs, learning-based methods have further been explored to address tier partitioning and congestion challenges, including TP-GNN [22], Snap-3-D [23], and recent ML-based 3D optimization frameworks [24], [25].

RL for PD: RL formulates PD optimization as a sequential decision process, where an agent iteratively selects design actions based on observed design states to optimize long-term objectives. RL-Sizer [26] applies deep RL to timing optimization, while TransSizer [27] introduces transformer-based architectures to model gate sizing decisions. Beyond sizing, RL has been applied to other optimization tasks, such as macro placement, where Mirhoseini et al. [28] formulate placement as an RL problem. RL-CCD [29] further

explores RL for clock skew scheduling by learning endpoint-specific optimization strategies. In addition, FastTuner [30] extends RL-based approaches to PD parameter optimization, enabling efficient exploration of high-dimensional design spaces.

Differentiable Optimization for PD: Differentiable optimization reformulates PD objectives into smooth and differentiable surrogates, enabling gradient-based optimization over design variables. Conventional PD optimization often relies on greedy heuristics or rule-based methods with limited ability to reason about global tradeoffs across objectives [31]. By contrast, differentiable optimization allows multiple design objectives to be jointly optimized in a unified end-to-end framework, where gradients provide direct guidance on how local design changes impact global quality metrics. As a result, differentiable optimization has been increasingly adopted in PD. Guo and Lin [32] introduced differentiable timing-driven placement by constructing smooth approximations of timing metrics, enabling gradient propagation through the placement process. For routability optimization, proxy metrics such as cell inflation [33] and RUDY-based estimation [34] have been widely used as differentiable surrogates for routing congestion. INSTA [1] introduces a GPU-accelerated differentiable STA

engine with 0.9999 correlation to commercial signoff tools, enabling large-scale gradient-based timing optimization by computing timing gradients with respect to arbitrary design variables.

The following sections discuss several representative works in greater detail, focusing on their key ideas and technical contributions.

GPU-Accelerated Timing Analysis

Efficient AI-driven PD relies on rapid and accurate timing feedback to support iterative optimization. While commercial signoff tools provide high accuracy, their runtime is prohibitive for the large number of iterations required by learning-based methods. STA exhibits structured level-wise parallelism, where timing arcs at the same topological level can be evaluated concurrently, which can be effectively exploited by GPU acceleration.

INSTA: Differentiable GPU-STA

Prior GPU-accelerated STA efforts have struggled to achieve industrial adoption because they attempt to build standalone timing engines that cannot replicate the proprietary delay models of commercial tools. INSTA adopts a fundamentally different philosophy, illustrated in Figure 1; rather than replacing commercial tools, it *complements* them by performing a one-time initial-

ization from any reference signoff engine, extracting arc delays, Synopsys Design Constraints (SDC) timing constraints, and on-chip variation (OCV) parameters.

The core design principle of INSTA is to achieve tool-accurate GPU-accelerated timing analysis while enabling gradient-based optimization over various leaf variables. Key inputs include cell and net arc attributes with respect to rise/fall conditions and unateness; timing exceptions, including multicycle and false paths; and start point/end point attributes, including clock arrival times and minimum required times. These inputs are extracted via custom TCL scripts into the CircuitOps [35] format—even on million-gate industrial designs, this extraction completes in approximately 10 min.

CUDA Kernel Implementation:

Figure 2 illustrates the CUDA kernel structure for timing propagation. The forward kernel computes arrival times level by level, where all arcs at the same timing level are processed in parallel by individual CUDA threads. The backward kernel then propagates gradients from endpoints back through the timing graph, computing the sensitivity of global timing metrics [worst negative slack (WNS)/TNS] with respect to each arc delay.

OCV Consideration: In modern industrial PD flows, STA engines must account for OCV to ensure reliable chip operation under extreme PVT conditions. INSTA models the arrival time at each pin as a Gaussian distribution characterized by

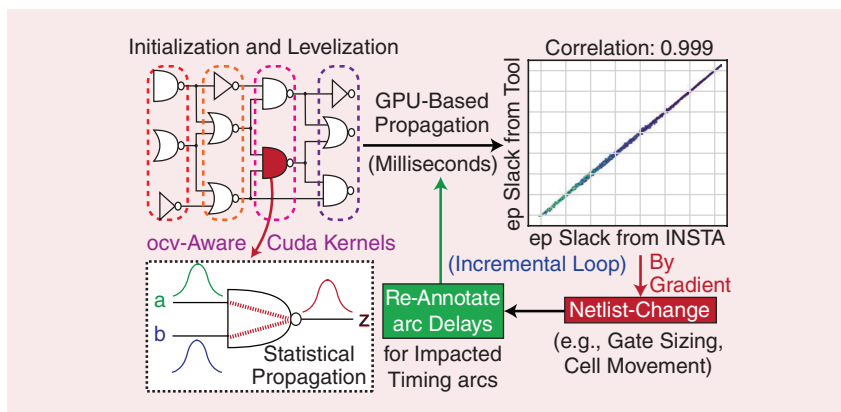


FIGURE 1: An overview of INSTA's initialization and application workflow. INSTA performs one-time initialization from commercial signoff tools, extracting arc delays, SDC constraints, and OCV parameters. This enables three key applications: timing evaluation, gradient-based gate sizing (INSTA-Size), and timing-driven placement (INSTA-Place).

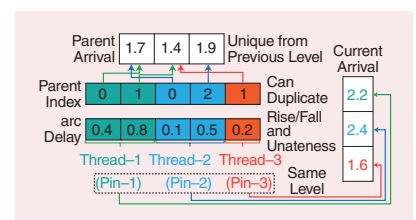


FIGURE 2: CUDA kernel implementation for GPU-accelerated timing propagation. The forward kernel computes arrival times level-by-level, while the backward kernel propagates gradients from endpoints to all timing arcs. Each CUDA thread processes arcs at the same timing level in parallel.

mean and standard deviation. For an arc $j \rightarrow i$ with variation attributes $\text{arc_mean}_{j \rightarrow i}$ and $\text{arc_std}_{j \rightarrow i}$, the final arrival time at sink i is

$$\text{arrival_mean}_i = \text{arrival_mean}_j + \text{arc_mean}_{j \rightarrow i} \quad (1)$$

$$\text{arrival_std}_i^2 = \text{arrival_std}_j^2 + \text{arc_std}_{j \rightarrow i}^2 \quad (2)$$

$$\text{arrival}_i = \text{arrival_mean}_i + N_\sigma \cdot \text{arrival_std}_i \quad (3)$$

where N_σ controls the level of pessimism (typically 3.0 in alignment with commercial signoff tools).

Common Path Pessimism Removal (CPPR) Handling via Priority Queues: CPPR is essential for accurate timing analysis as it mitigates pessimism introduced by clock paths [36]. In INSTA, CPPR is addressed through an efficient GPU-

accelerated method that employs Top-K arrival time propagation while ensuring that each arrival distribution is mapped to a distinct start point. Priority queues are dynamically maintained at each pin to track the Top-K critical arrival distributions with respect to unique timing start points.

Enabling Differentiability: The “greater than” operation in standard STA identifies the maximum arrival time from multiple incoming paths but introduces nondifferentiability. To overcome this, INSTA adopts the numerically stable log-sum-exponential (LSE) operator

$$\text{LSE}(\{A_i\}_{i=1}^n) = M + \tau \cdot \log\left(\sum_{i=1}^n \exp\left(\frac{A_i - M}{\tau}\right)\right) \quad (4)$$

where $M = \max(A_i)$ ensures numerical stability, and $\tau > 0$ controls

smoothness. The gradient acts as a softmax-like attention mechanism, naturally assigning higher gradients to timing-critical paths as follows:

$$\frac{\partial \text{LSE}}{\partial A_i} = \frac{\exp\left(\frac{A_i - M}{\tau}\right)}{\sum_{j=1}^n \exp\left(\frac{A_j - M}{\tau}\right)}. \quad (5)$$

For the first time in PD, INSTA introduces the concept of “timing gradients” to enable arc-based and truly global timing optimization (Figure 3). The timing gradient of each arc precisely quantifies its contribution to global timing metrics such as WNS and TNS, enabling targeted and fine-grained optimization. If we draw an analogy between INSTA and modern ML models, each pin corresponds to a neuron, and arc delays correspond to weights—INSTA computes timing metrics in a fully differentiable manner.

Table 1 demonstrates INSTA's exceptional correlation with an industry-leading signoff tool across five industrial designs in a commercial 3-nm node. The subsecond runtime enables thousands of timing evaluations per minute on a single GPU. Notably, since GPU parallelism amortizes the computation at each timing level to $O(1)$, INSTA's runtime scales with the number of timing levels under a fixed Top-K rather than the total number of pins or cells.

INSTA Applications

Timing Evaluation in Commercial Sizing Flow: INSTA serves as a timing evaluator within commercial gate sizing flows for timing-constrained power optimization. Combined with the tool's EstimateEco command for arc delay reannotation, INSTA delivers a **14x** speedup over highly optimized in-house STA engines and **25x** runtime improvement over commercial signoff tools while maintaining extremely high correlation throughout the optimization process.

INSTA-Size for Timing Optimization: Using timing gradients,

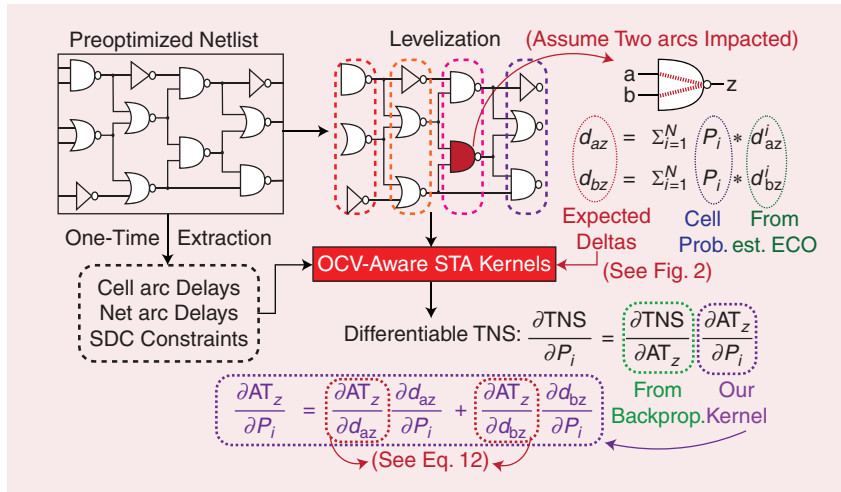


FIGURE 3: A differentiable STA engine for LLM-predicted gate size probability refinement. Arc delays are reannotated using the dot product between LLM-predicted probabilities (Prob.) and delta delay estimates from estimateECO (est. ECO). The timing gradient quantifies each arc's contribution to global TNS. Backprop.: backpropagation.

TABLE 1. INSTA CORRELATION WITH COMMERCIAL SIGNOFF TOOL (3 NM).

DESIGN	NO. OF CELLS	NO. OF PINS	CORR.	RT(s)	MEM
Block-1	4M	15M	0.99994	0.39	21 GB
Block-2	2M	6M	0.99999	0.35	6 GB
Block-3	3M	9M	0.99992	0.37	12 GB
Block-4	2M	7M	0.99996	0.35	9 GB
Block-5	2M	6M	0.99999	0.33	6 GB

M: million.

INSTA-Size achieves up to 15% improvement in TNS compared to commercial signoff tools' default timing optimization engines while requiring 42% fewer cell modifications on average across the IWLS benchmark suite [37]. The key insight is that timing gradients provide instant identification of the most critical stages and their corresponding sensitivities to TNS.

INSTA-Place for Timing-Driven Placement: INSTA-Place achieves up to 59.4% TNS improvement and 16.2% wirelength reduction compared to DREAMPlace 4.0 [5], the state-of-the-art net weighting-based timing-driven placer, by addressing fundamental shortcomings through arc-based timing gradient weighting.

LLM-Assisted Netlist Optimization

The remarkable success of LLMs stems from their ability to learn rich and transferable representations from massive datasets [38], [39]. A key insight of Lib2Vec [2] is that similar principles apply to IC design; netlists can be viewed as structured sequences, library cells as vocabulary tokens (Figure 4), and timing optimization as sequence transformation.

Lib2Vec: Learning Universal Cell Representations

Modern technology libraries contain tens of thousands of cells with diverse characteristics. Traditional approaches represent cells using handcrafted features or one-hot encodings, which fail to capture semantic relationships [35]. Lib2Vec addresses this through self-supervised learning from Liberty files.

The key insight is that a cell's semantics are defined by its *responses to input conditions*—cells exhibiting similar input-output relationships should have similar representations. The architecture employs attention mechanisms [39], [40] to handle varying pin counts; for functional output prediction, an attention layer generates the functional embedding for an output pin by attending to the cell's functional embedding and em-

beddings of all corresponding pins. For electrical output prediction, a property-specific cell representation is created by concatenating the base electrical embedding with a property token embedding (e.g., rise delay). An attention layer then combines this with input/output pin embeddings to create timing arc embeddings.

Since functional properties are independent of electrical properties, two separate models learn functional and electrical representations, respectively, though they share a similar attention-based architecture. This design enables Lib2Vec to produce consistent-length vector representations for cells with different pin counts while supporting property-specific representations for both entire cells and individual timing arcs.

LEGO-Size: Gate Sizing as Language Modeling

LEGO-Size, illustrated in Figure 5, reformulates gate sizing as a language modeling task. The key innovation is *tokenization* (Figure 4); each library-cell name is decomposed into three semantic tokens representing the gate type, drive strength, and threshold voltage. While a 3-nm library contains more than 10,000 cells, they can be characterized by approximately 300 tokens as follows:

$$|\text{LibCells}| = |\text{Types}| \times |\text{Drives}| \times |V_{th}|. \quad (6)$$

The sum of tokens across the three sets is much smaller than the total number of combinations, dramatically reducing vocabulary size

while preserving semantic meaning. Additionally, a “function token” per library-cell group denotes the functionality—only library cells with the same function are swappable.

Self-Supervised Pretraining:

Word embeddings in natural language processing (NLP) require pretraining for effective downstream tasks [38]. LEGO-Size employs two self-supervised tasks illustrated in Figure 6: 1) masked token prediction, where tokens are randomly masked and the model reconstructs them using context, and 2) cell-based arrival time increment prediction, where the model predicts stage delays using CNN-encoded timing tables. By pretraining the embeddings with PD-aware self-supervised tasks, LEGO-Size [41] demonstrates significantly better generalization on unseen designs than previous works [26], [27] that rely on direct supervised finetuning for gate sizing prediction.

Netlist Encoding via Graph

Transformer (GT): While LEGO-Size follows path-by-path optimization like commercial tools, graph-based netlist attributes are vital. The decision on which cells to resize depends on factors such as the number of violating paths a cell sits on, its fan-out/fan-in structure, and sensitivity to other endpoints. As shown in Figure 7, we employ a GT model [40] with linear-complexity attention from Performer [42], approximating softmax with kernel feature maps

$$\text{Linear-Attention}(Q, K, V) = \phi(Q)(\phi(K)^T V) \quad (7)$$

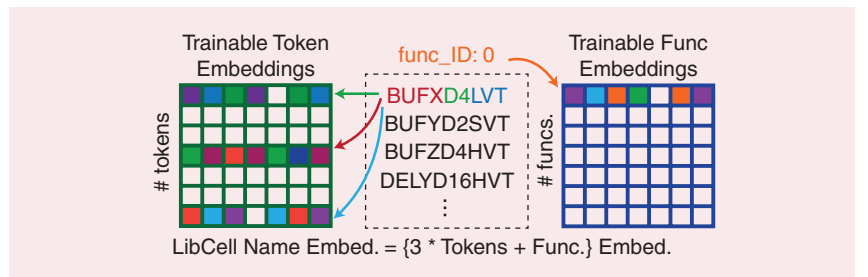


FIGURE 4: Library-cell embedding construction through token and function embeddings (Func. Embed.). Each library-cell name is tokenized into three distinct tokens: gate type (red), drive strength (green), and V_{th} type (blue). A function token (func_ID) captures the cell's logical functionality—only cells with the same function are swappable.

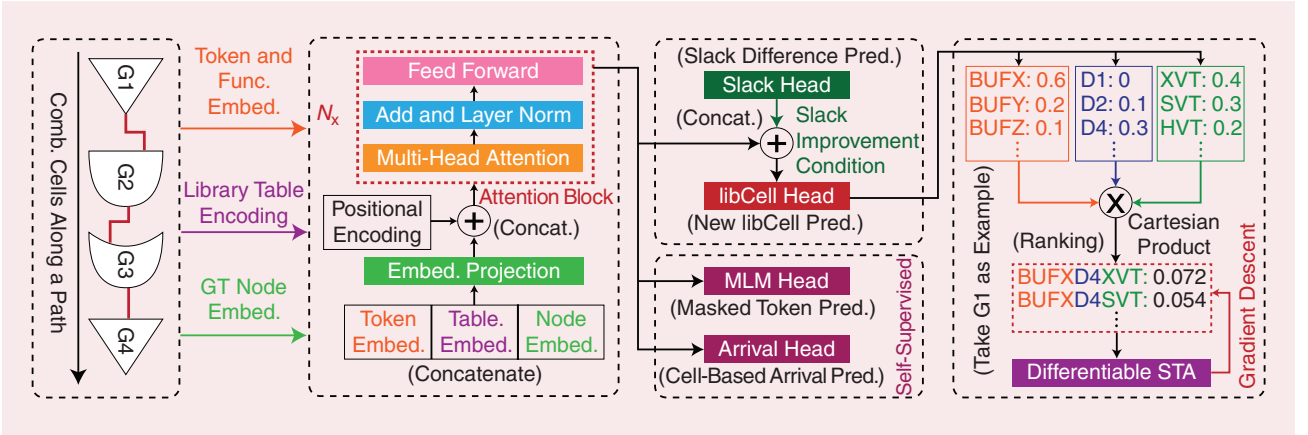


FIGURE 5: The detailed architecture of LEGO-Size. A slack improvement conditioning mechanism is introduced to the gate size prediction head (libCell head), which is followed by a differentiable STA engine for refinement of the LLM-predicted probabilities of library cells to achieve better-than-tool optimization results. Pred.: prediction; Concat.: concatenate.

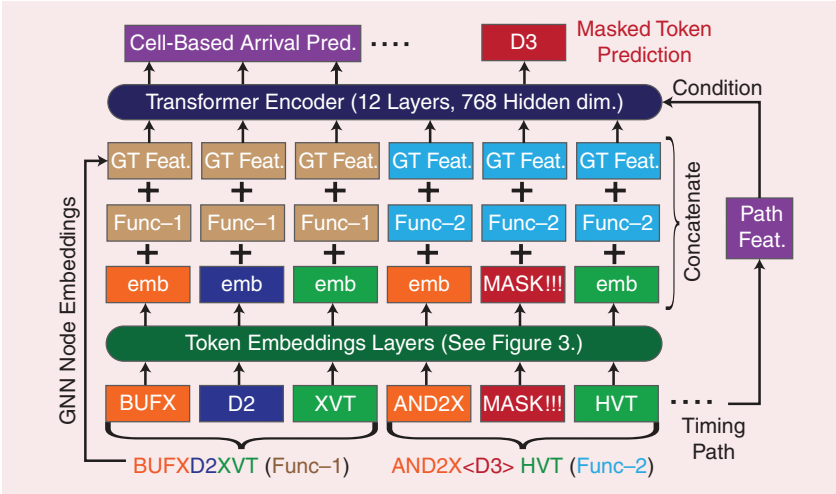


FIGURE 6: Self-supervised pretraining tasks in LEGO-Size: (1) masked token prediction where tokens are randomly masked and the model reconstructs them using context, and (2) cell-based arrival time increment prediction where the model predicts stage delays. "GT Feat." refers to node embeddings from the Graph Transformer (GT) model. Emb.: embeddings.

reducing complexity from $O(n^2)$ to $O(n)$ and enabling efficient computation on million-cell netlists. The aggregation process at each layer combines the graph isomorphism network with edge features. for local aggregation and linear attention for global netlist encoding as follows:

$$X_{MP}^{(l+1)}, E^{(l+1)} = \text{GINE}(X^{(l)}, E^{(l)}, A) \quad (8)$$

$$X_{LA}^{(l+1)} = \text{Linear-Attention}(X^{(l)}) \quad (9)$$

$$X^{(l+1)} = \text{MLP}(X_{MP}^{(l+1)} + X_{LA}^{(l+1)}). \quad (10)$$

Cross-Attention for Constraints:

Unlike conventional NLP tasks, gate sizing is a sequence-to-sequence problem where each input token maps directly to a corresponding

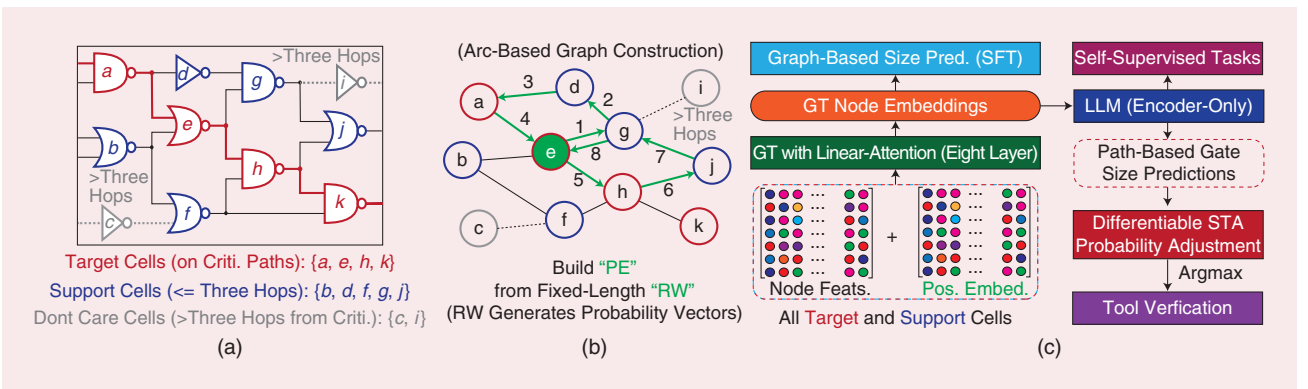


FIGURE 7: Node embedding construction via scalable GT with linear-complexity attention. (a) Definition of target cells (on PBA-violating paths), support cells (within three-hop neighborhood), and don't care cells. (b) Positional embeddings (Pos. Embed.) derived from random walks. (c) An eight-layer GT generates task-relevant node embeddings through a graph-based gate sizing prediction task. PE: positional embeddings; RW: random walk.

output token governed by rules. As shown in Figure 8, cross-attention between each input token and its permitted tokens, governed by a constraint mask, enables effective embedding learning while ensuring that illegal tokens receive zero scores. This mechanism can handle complex cell constraint rules—for example, limiting buffer-function cells to “delay cells” only for hold optimization.

Slack-Conditioned Prediction:

The model learns the precise gate size change required to achieve a target slack improvement (the condition), enabling flexible tradeoffs between timing and power. During supervised fine-tuning, teacher forcing uses ground-truth slack improvement values; at inference, predicted values drive the model. An entropy term serves as regularization loss alongside the standard cross-entropy loss

$$\mathcal{L}_{\text{size}} = \text{CE}(\hat{Y}_{\text{size}}, Y_{\text{size}}) + \lambda \cdot H(\hat{Y}_{\text{size}}) \quad (11)$$

$$\mathcal{L}_{\text{slack}} = \text{RMSE}(\hat{Y}_{\text{slack}}, Y_{\text{slack}}) \quad (12)$$

preventing overconfidence and ensuring better generalization.

INSTA-Guided Refinement:

The final component is INSTA’s differentiable STA engine that refines LLM-predicted gate size probabilities. For each cell, given sizing candidates and LLM-predicted probabilities, arc delays are reannotated by computing the dot product between probabilities and delay estimates from the commercial tool’s `estimateECO` command. To account for the assumption that neighboring cells remain unchanged, gates are prioritized by the cumulative slack of path-based analysis-violating paths they lie on, and a subgraph blocking mechanism prevents gates with overlapping subgraphs from being selected in the same iteration. The TNS gradient then updates probabilities through gradient descent, achieving better-than-tool results.

Table 2 presents results on five *completely unseen* industrial designs—designs never used during training. LEGO-Size consistently outperforms the commercial signoff tool,

achieving an average 26% TNS reduction with more than 100× speedup.

BUFFALO: Single-Shot Buffer Tree Generation

Buffer insertion is another fundamental PD task traditionally solved through dynamic programming or greedy heuristics. BUFFALO reformulates this as sequence-to-sequence generation using a T5 encoder-decoder architecture [43] (Figure 9).

Large Buffering Dataset Generation:

To ensure robust model generalization, we designed an automated end-to-end pipeline that generates more than 20 million paired examples of unbuffered and buffered nets. An RTL net generator synthesizes diverse and high-fan-out netlists under realistic constraints, including variations in driver and sink placements, fan-out counts (1–100), delay targets (100 ps–1 ns), input transitions, and

input delays. A commercial PD tool then inserts buffers while adhering to realistic timing constraints. Each resulting unbuffered and buffered net is serialized into a structured depth-first search (DFS) bracketed sequence.

Full-Tree Tokenization:

Instead of unique tokens per cell, we adopt factorized tokenization, splitting each library-cell name into gate type and size tokens, exploiting the Cartesian structure of standard-cell libraries. This significantly reduces vocabulary size and enables efficient parameter sharing among cells of similar attributes. Special tokens encode tree structures: `<(>` and `<)>` for hierarchy, plus standard delimiters.

Dual-Head Decoding:

At each decoding step, the decoder’s hidden state feeds two parallel heads: a structural head selecting the next token (a cell identifier or structural symbol) and a location head predicting

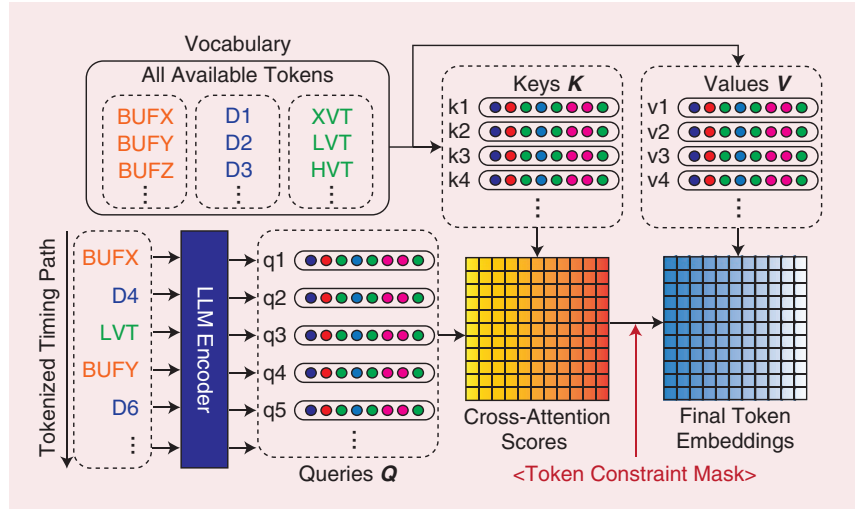


FIGURE 8: Cross-attention mechanism for effective embedding learning and constraint handling. Each input token (query) attends to its permitted output tokens (keys/values) governed by a constraint mask, ensuring illegal tokens receive zero scores.

TABLE 2. LEGO-SIZE VERSUS COMMERCIAL SIGNOFF TOOL (3 NM, UNSEEN DESIGNS).

DESIGN	CELLS	TNS IMP.	NVE IMP.	SPEEDUP
Block-1	1.8M	–36.7%	–47.3%	125×
Block-2	1.3M	–30.0%	–27.4%	111×
Block-3	1.2M	–19.5%	–10.4%	100×
Block-4	1.5M	–17.8%	–15.3%	119×
Block-5	1.4M	–27.4%	–17.2%	114×
Average		–26.3%	–23.5%	114×

buffer coordinates (meaningful only at buffer size token positions). The composite loss combines structural classification and masked coordinate regression

$$\mathcal{L} = \sum_i \left[-\log p(\text{token}_i) + \lambda m_i \|\hat{\mathbf{I}}_i - \mathbf{I}_i\|_2^2 \right] \quad (13)$$

where mask m_i activates coordinate loss only at buffer size positions.

Group Relative Policy Optimization (GRPO) Refinement: While supervised fine-tuning imitates reference solutions (Figure 10), it treats all solutions as equally optimal without balancing PPA tradeoffs. GRPO [44] evaluates multiple candidate buffer trees per net and optimizes based on relative PPA quality using Group Relative Advantage Estimation

$$A^{(g)} = w_1 \frac{\Delta \text{TNS}^{(g)} - \mu_{\text{TNS}}}{\sigma_{\text{TNS}}} + w_2 \frac{-\Delta \text{Area}^{(g)} - \mu_{\text{Area}}}{\sigma_{\text{Area}}} \quad (14)$$

where each objective is standardized independently. GRPO also enforces structural legality by penalizing outputs that violate predefined structural requirements—any candidate failing these criteria is assigned a fixed z-score advantage of -3 . INSTA provides rapid timing evaluation for the multicandidate feedback.

Chip-Level GRPO: To jointly coordinate buffering decisions across multiple critical nets simultaneously, we identify the most timing-critical nets using INSTA.netGrad (Figure 11), which quantifies each net's contribution to total timing delay, and apply our trained buffering policy collectively in a unified GRPO update. On nine designs at 7 nm, BUFFALO achieves 71% TNS and 67% WNS improvement with 83× speedup.

RL for Design Automation

While supervised learning mimics existing solutions, RL enables the *dis-*

covery of strategies that surpass the training distribution by optimizing directly for the end objective [44].

RL-CCD: Intelligent Endpoint Prioritization

Concurrent Clock and Data (CCD) optimization is a critical technique in modern PD flows that jointly adjusts clock skew and data path delays to improve timing [29], [45]. However, commercial tools treat all violating endpoints uniformly, missing that *different endpoints respond differently to clock versus data optimization*.

RL-CCD (Figure 12) learns to identify which endpoints benefit most from useful skew. Given a globally placed netlist $G = (V, E)$, the goal is to select violating endpoints $V' \in V$ to be prioritized for useful skew optimization such that post-placement TNS is optimized. The key idea is to let the useful skew engine “over-fix” the timing of RL-selected endpoints so that

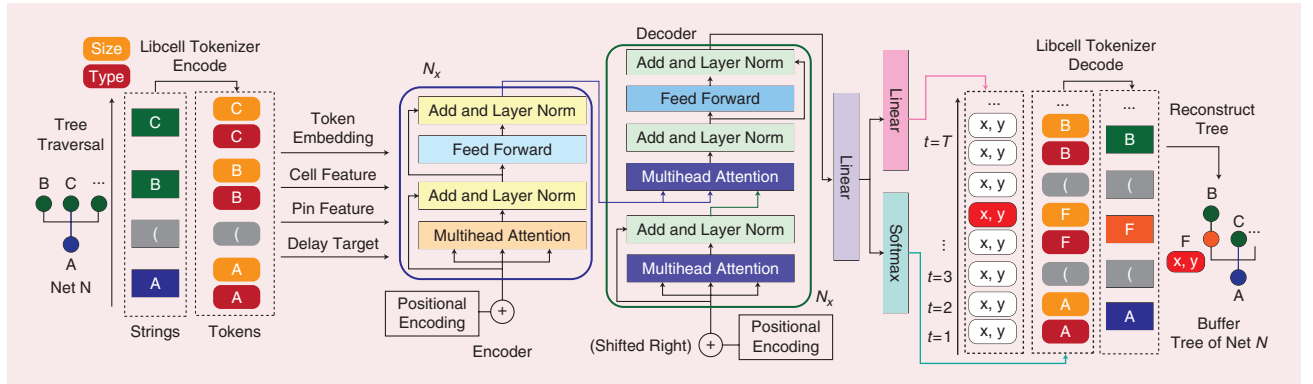


FIGURE 9: The detailed architecture of BUFFALO's T5-based encoder-decoder model. The encoder processes tokenized unbuffered nets with cell embeddings, while the decoder employs dual-head decoding; a structural head selects tokens (cell identifiers or structural symbols), and a location head predicts buffer coordinates at buffer size token positions.

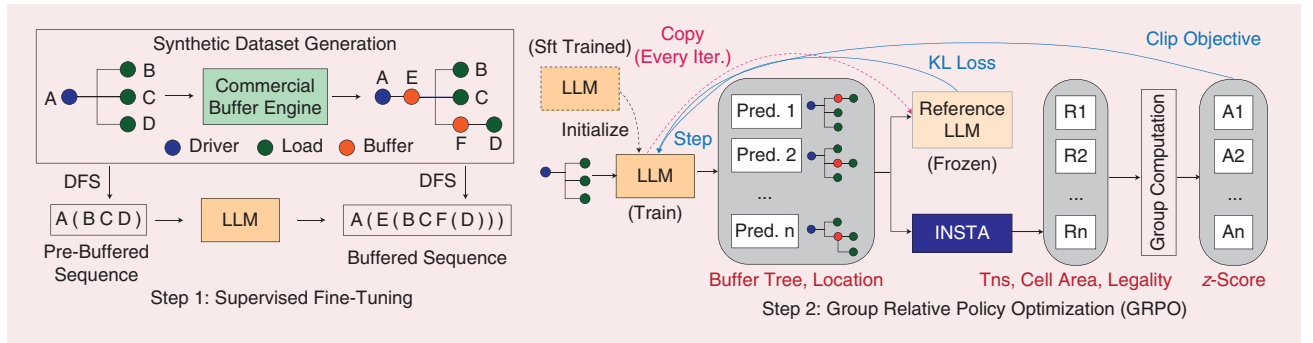


FIGURE 10: The overall BUFFALO training workflow. Supervised fine-tuning (SFT) trains the model to imitate reference solutions from commercial tools. Iterative GRPO policy refinement then optimizes for PPA objectives using multicandidate feedback from INSTA timing evaluation. KL = Kullback-Leibler.

subsequent delay optimization techniques can spend less effort on them, achieving flow-wise optimal solutions.

Markov Decision Process (MDP) Formulation: The endpoint prioritization problem is formulated

as an MDP where the following conditions apply:

- **States (s):** A state s_t represents the status of the endpoint set $EP \in V$, encoded by the proposed EP-GNN framework via fan-in cone aggregation.

- **Actions (a):** An action a_t refers to the endpoint being selected.
- **Reward (r):** Rewards are zero for intermediate actions except for the last action, which represents the final achieved TNS value after the entire placement optimization.

The goal is to obtain an optimal policy π that maximizes the expected return as follows:

$$\max_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]. \quad (15)$$

EP-GNN Encoding: The Endpoint-GNN framework (Figure 13) generates node embeddings through iterative neighborhood and fan-in cone aggregation [46], [47]. With three graph convolution layers and one fully connected layer, each layer transforms node features as follows:

$$f_v^l = \sigma \left(\gamma f_v^{l-1} \cdot \Theta_{\text{proj}} + (1 - \gamma) \cdot \Theta_{\text{agg}} \left(\frac{1}{|N(v)|} \sum_{j \in N(v)} f_j^{l-1} \right) \right) \quad (16)$$

FIGURE 11: The chip-level GRPO strategy. Critical nets are identified using INSTA.netGrad, which quantifies each net's contribution to total timing delay. The trained buffering policy is then applied collectively to high-impact nets in a unified GRPO update for global timing improvement.

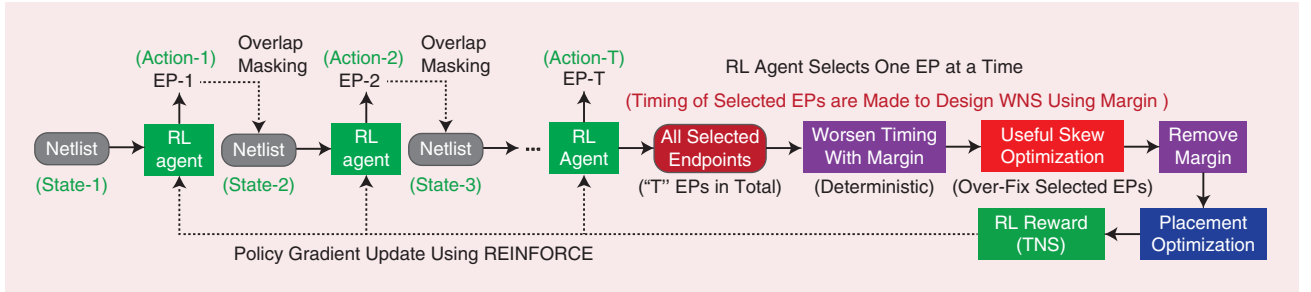


FIGURE 12: A high-level overview of RL-CCD framework. At each RL time step, the agent selects one endpoint and masks out overlapping endpoints (EP). Selected endpoints receive timing margins before useful skew optimization, enabling them to be “over-fixed” by clock arrival adjustments. The final TNS serves as the RL reward for REINFORCE parameter updates.

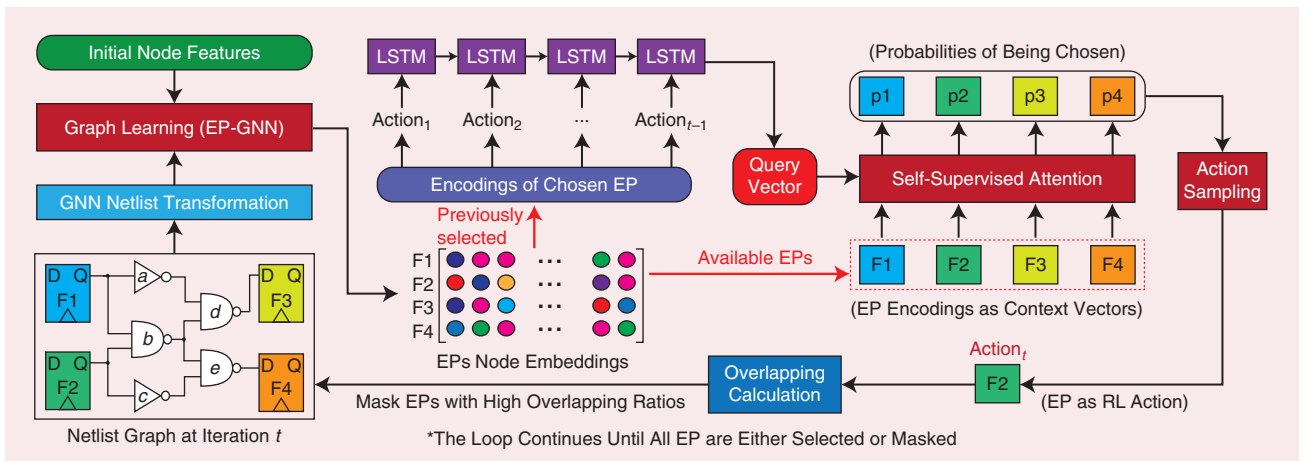


FIGURE 13: The detailed RL-CCD endpoint selection process. EP-GNN generates endpoint embeddings through fan-in cone aggregation (state S_t). A long short-term memory (LSTM) encodes past actions sequentially, producing query vector q_t for the attention-based decoder. The decoder outputs probability distribution P_t over all endpoints for sampling action a_t . Overlap masking then removes endpoints with high fan-in cone overlap.

where γ weighs self-projection versus neighborhood aggregation. The final endpoint embedding aggregates fan-in cone information.

$$f_e = \Theta_{FC} \left(f_e^{l=3} + \sum_{j \in \text{cone}(e)} f_j^{l=3} \right). \quad (17)$$

Attention-Based Action Decoding: Using a self-supervised atten-

tion mechanism [39], the decoder takes the query vector and current endpoint embeddings as inputs and outputs a probability vector

$$A_t^{(i)} = \begin{cases} v^T \tanh(W_1 \cdot F_{EP}^{(i)} + W_2 \cdot q_i) & \text{if valid} \\ -\infty & \text{otherwise} \end{cases} \quad (18)$$

followed by softmax to compute selection probabilities.

Fan-in Cone Overlap Masking:

After each selection, endpoints with a fan-in cone overlap ratio above threshold ρ are masked (Figure 14) to prevent conflicting clock adjustments. This allows the RL agent to adaptively decide the total number of endpoints based on design characteristics. Table 3 shows consistent improvements across 19 industrial designs spanning from 5 to 12 nm, with up to 64% TNS improvement in the best case.

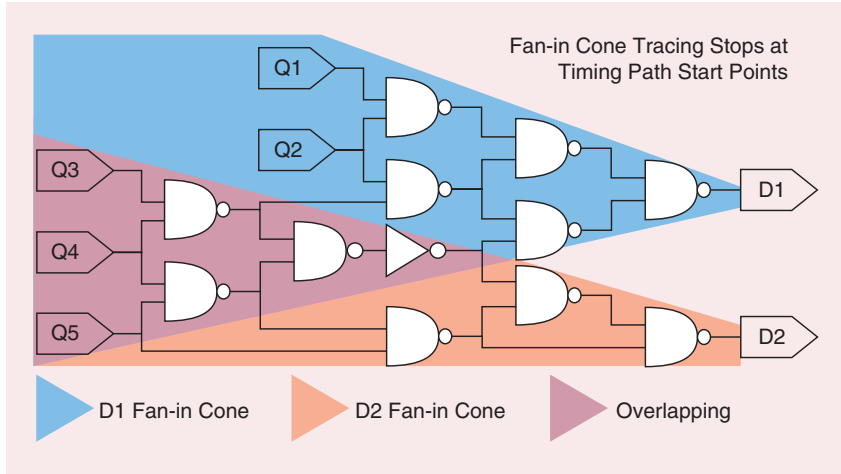


FIGURE 14: Endpoint fan-in cone overlapping calculation. Fan-in cone tracing stops at previous start points. The overlapping ratio is computed by dividing the number of overlapped cells by the total fan-in cone cells. Endpoints with an overlap ratio above threshold ρ are masked to prevent conflicting clock adjustments.

TABLE 3. RL-CCD RESULTS ACROSS TECHNOLOGY NODES.

NODE	DESIGNS	TNS IMP.	NVE IMP.	POWER
5 nm	6	28.3%	22.1%	-2.1%
7 nm	8	21.5%	17.8%	-1.8%
12 nm	5	19.2%	15.4%	-1.5%
All	19	23.0%	18.8%	-1.8%
Best case		64.0%	66.5%	-4.2%

FastTuner: Automated Design Space Exploration

Modern commercial PD tools offer hundreds of tunable parameters. Traditional tuning methods, including Bayesian optimization and evolutionary algorithms, require full place and route runs to evaluate each configuration—prohibitively expensive for large designs.

FastTuner formulates parameter tuning as an MDP where the state comprises netlist features plus current parameters, actions are parameter selections, and rewards are PPA metrics from offline-trained estimators. The key innovation is *instant reward approximation*; GNN [47] encodes the netlist, and a Transformer encoder-decoder [39] sequentially predicts PD parameters with instant PPA feedback from pretrained estimators, eliminating hours-long placement and routing feedback.

The RL tuner utilizes a GNN to capture underlying design features, enabling generalization to unseen designs. A pretrained FastTuner can significantly improve optimization results with just a few iterations of fine-tuning. Across seven designs at 28

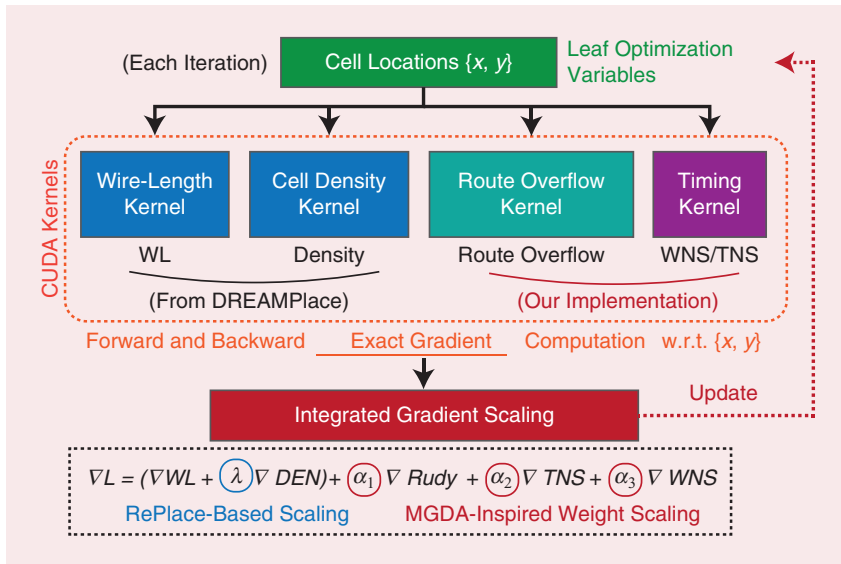


FIGURE 15: An overview of C3PO's exact, differentiable multiobjective optimization. A convex-based weight scaling technique automatically determines objective weights at each iteration, concurrently optimizing timing, routability, and wirelength. w.r.t.: with respect to; WL = wirelength.

nm, FastTuner achieves up to 79% TNS improvement and 12% power reduction with 50× speedup over traditional design space exploration methods.

Differentiable Optimization for Placement

C3PO: Commercial-Quality GPU Placement

Despite significant progress, GPU-based placers have struggled to match commercial tool quality due to reliance on indirect net-weighting for timing [5] and cell inflation for routability [33]. C3PO (Figure 15) bridges this gap through concurrent multiobjective optimization with exact gradient computation.

Differentiable STA Engine: As shown in Figure 16, C3PO integrates fully differentiable delay calculations directly into GPU kernels with cell locations as leaf variables. The delay computation includes: 1) a custom FLUTE-based [31], [48] kernel for Steiner point insertion with gradient redistribution to surrounding non-Steiner nodes; 2) an Elmore-based kernel for net delay via post-order DFS and preorder BFS sweeps; and 3) a solver-based kernel for cell delay using polynomial fits that handle out-of-range load-slew queries gracefully.

For Steiner point gradient redistribution, we push each Steiner point's gradient to its surrounding non-Steiner nodes, including those on diagonals

$$\Delta x_n = w_x(n, s) \text{sgn}(x_s - x_n) |\nabla x_s| \quad (19)$$

$$\Delta y_n = w_y(n, s) \text{sgn}(y_s - y_n) |\nabla y_s| \quad (20)$$

where horizontal neighbors receive Δx_n , vertical neighbors receive Δy_n , and diagonal neighbors receive both.

For cell delay, we adopt a solver-based evaluation that stays accurate for arbitrary load-slew pairs. Given query values (ℓ, s) , we locate bracket indices, fit a polynomial $Z(\ell, s) = A + B\ell + Cs + D\ell s$ using the four corner nonlinear delay model (NLDM) values, and directly compute derivatives as follows:

$$\frac{\partial Z}{\partial \ell} = B + Ds, \quad \frac{\partial Z}{\partial s} = C + D\ell. \quad (21)$$

Differentiable Routability: C3PO develops the first fully differentiable RUDY kernel with exact gradients (Figure 17). Using chain and quotient rules

$$\begin{aligned} \frac{\partial \text{RUDY}_n}{\partial x_p} &= \frac{w_n}{y_{\text{span}}} \frac{\partial \text{OV}_n}{\partial x_p} + \frac{w_n}{x_{\text{span}}^2} \\ &\times \left[x_{\text{span}} \frac{\partial \text{OV}_n}{\partial x_p} - \text{OV}_n \frac{\partial x_{\text{span}}}{\partial x_p} \right] \end{aligned} \quad (22)$$

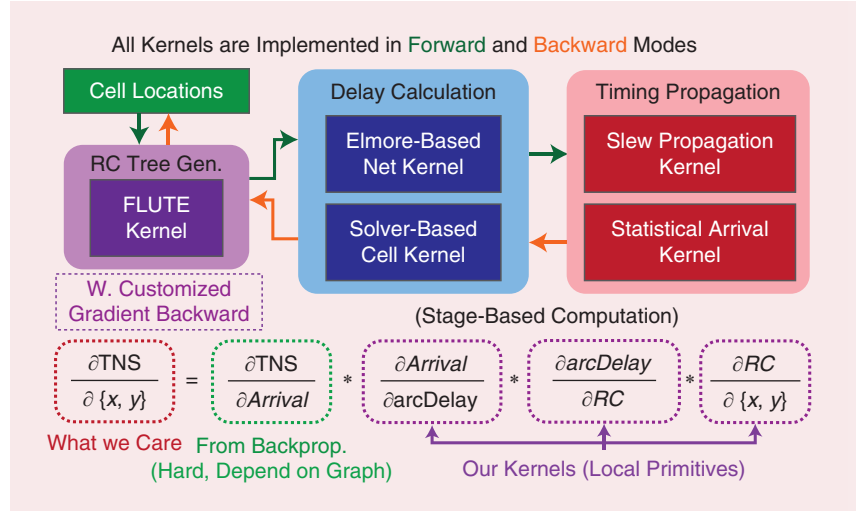


FIGURE 16: C3PO timing kernels for end-to-end differentiable TNS/WNS computation. Custom CUDA kernels implement: (1) FLUTE-based Steiner point insertion with gradient redistribution, (2) Elmore-based net delay via DFS/BFS sweeps, (3) solver-based cell delay using polynomial fits, and (4) timing propagation with LSE-based soft max. Tree Gen.: tree generation; w: with.

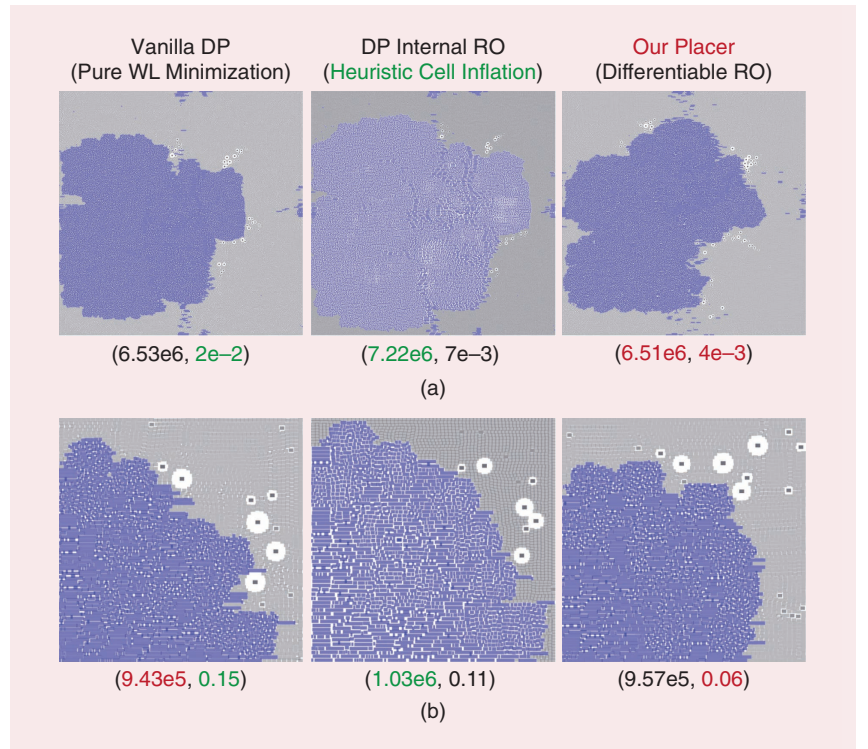


FIGURE 17: A comparison of routability optimization methods. C3PO's differentiable RUDY kernel achieves up to 45.4% better RouteOverflow than cell inflation with negligible wirelength overhead, avoiding the whitespace creation that degrades wirelength. (a) Design: FPU. (b) Design: DES; DP = DREAMPlace; RO = routability optimization.

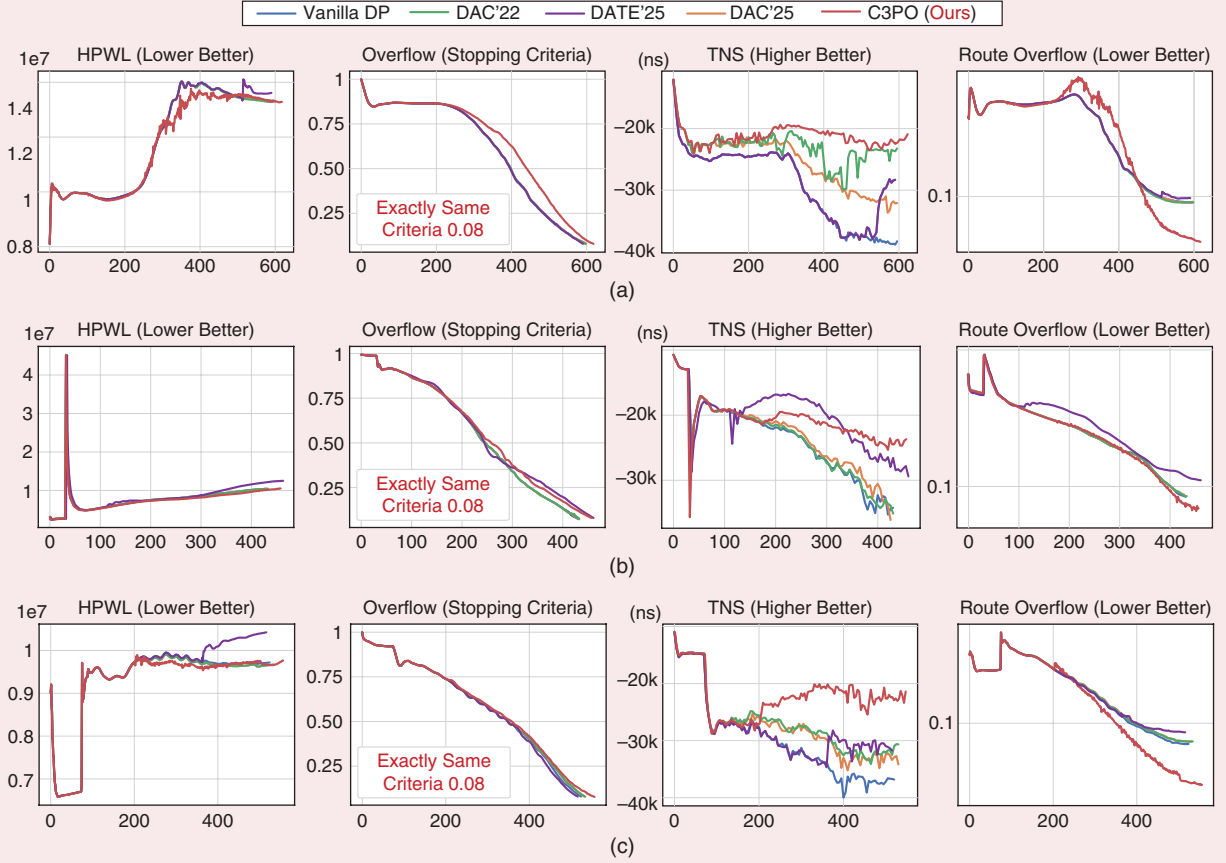


FIGURE 18: A comparison between C3PO and state-of-the-art GPU-based timing-driven placers on TILOS benchmarks [(a) MEMPOOL, (b) ARIANE133, and (c) ARIANE136]. All methods use identical stopping criteria of 0.08 overflow. C3PO achieves superior TNS, wirelength, and routability through concurrent multiobjective optimization. HPWL = half-perimeter wirelength.

TABLE 4. C3PO VERSUS COMMERCIAL PLACEMENT (POST-ROUTE RESULTS).

BENCHMARK	WL	TNS	POWER	CONG.
MEMPOOL	-10.2%	-31.5%	-13.8%	-7.1%
ARIANE133	-12.8%	-35.7%	-16.2%	-9.4%
ARIANE136	-11.9%	-34.4%	-15.3%	-8.1%
Average	-11.6%	-33.9%	-15.1%	-8.2%

capturing both overlap sensitivity and bounding box changes—terms neglected in prior work. The kernel reduces RouteOverflow by up to 45.4% relative to cell inflation with negligible wirelength overhead.

Multiple-Gradient Descent Algorithm (MGDA)-Based Weighting: Rather than heuristic weight tuning, C3PO employs a MGDA-inspired [49] scaling strategy. We fix $g_0 \equiv g_{WL+DEN}$ and seek nonnegative weights for remaining gradients

$$g_{\text{comb}} = g_0 + \sum_{i=1}^{K-1} \alpha_i g_i \quad (23)$$

where

$$\alpha_i = \omega_i^{(0)} \frac{\|g_0\|}{\|g_i\| + \varepsilon}. \quad (24)$$

A regularized linear system dynamically computes optimal secondary weights at each iteration, eliminating the need for manual parameter tuning while ensuring placement convergence.

Figure 18 and Table 4 demonstrate substantial improvements across all quality metrics on TILOS benchmarks, validating that optimizing timing directly surpasses proxy-based approaches. Figure 19 shows that the commercial validation flow ensures that the placement improvements translate to post-route benefits.

Differentiable 3D IC Optimization

Three-dimensional integration introduces unique challenges: tier assignment, inter-tier connections, and distinct congestion patterns across dies [22], [50], [51].

DCO-3-D: DCO-3-D [25] introduces a differentiable framework that incorporates routability awareness into 3D placement optimization. A Siamese U-Net is employed to predict post-route congestion for the top and bottom dies based on 3D

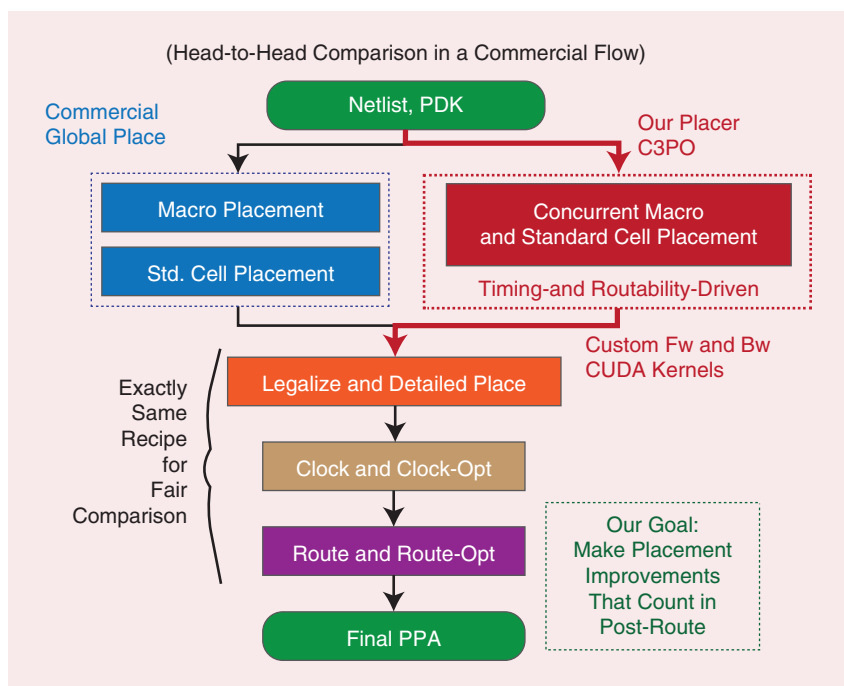


FIGURE 19: The commercial validation flow for fair comparison. C3PO replaces only global placement while using identical downstream optimization (detailed placement, CTS, and routing) to ensure placement improvements translate to post-route benefits. CTS = clock tree synthesis; PDK = process design kit.

placement features. The key innovation lies in 3D cell spreading, where a continuous variable $z \in [0, 1]$ represents the probability of spreading a cell to the top die. This probabilistic formulation allows each cell to contribute fractionally to both tiers during optimization, while a final hard assignment is obtained at inference time via $z = \mathbf{1}_{z \geq 0.5}$.

DRG-3-D: DRG-3-D [52] provides routing guidance through differentiable multiobjective optimization of overflow, wirelength, via cost, and face-to-face bonding cost. Soft routing selection with discretization yields a routing tier guide that specifies preferred routing layers per net, encouraging routers to confine most wirelength within designated tiers.

Conclusion

This article highlights a clear transition in PD from handcrafted rule-based heuristics toward learning-driven methodologies that better address the scalability and complexity of modern ICs. Advances in GPU acceleration, ML, LLMs, RL, and differentiable optimization demonstrate that AI techniques

can substantially improve scalability and solution quality when tightly integrated with PD workflows. Looking ahead, as algorithmic techniques and computing platforms continue to evolve, AI-assisted methods are expected to serve as core optimization engines that directly drive solution quality and runtime at scale.

References

- [1] Y.-C. Lu, Z. Guo, K. Kunal, R. Liang, and H. Ren, "INSTA: An ultra-fast, differentiable, statistical static timing analysis engine for industrial physical design applications," in *Proc. 62nd ACM/IEEE Des. Automat. Conf. (DAC)*, 2025, pp. 1–7, doi: [10.1109/DAC63849.2025.11132858](https://doi.org/10.1109/DAC63849.2025.11132858).
- [2] R. Liang, Y.-C. Lu, W.-H. Liu, and H. Ren, "Learning library cell representations in vector space," 2025, *arXiv:2503.22900*.
- [3] H.-H. Hsiao, Y.-C. Lu, S. K. Lim, and H. Ren, "BUFFALO: PPA-configurable, LLM-based buffer tree generation via group relative policy optimization," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2025, pp. 1–9, doi: [10.1109/ICCAD66269.2025.11240744](https://doi.org/10.1109/ICCAD66269.2025.11240744).
- [4] Y. Lin et al., "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 4, pp. 748–761, Apr. 2021, doi: [10.1109/TCAD.2020.3003843](https://doi.org/10.1109/TCAD.2020.3003843).
- [5] P. Liao, S. Liu, Z. Chen, W. Lv, Y. Lin, and B. Yu, "DREAMPlace 4.0: Timing-driven placement with momentum-based net weighting," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2022, pp. 939–944, doi: [10.23919/DATE54114.2022.9774725](https://doi.org/10.23919/DATE54114.2022.9774725).

- [6] Z. Guo, T.-W. Huang, and Y. Lin, "GPU-accelerated static timing analysis," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2020, pp. 1–9, doi: [10.1145/3400302.3415631](https://doi.org/10.1145/3400302.3415631).
- [7] S. Lin, J. Liu, E. F. Young, and M. D. Wong, "GAMER: GPU-accelerated maze routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 583–593, Feb. 2023, doi: [10.1109/TCAD.2022.3184281](https://doi.org/10.1109/TCAD.2022.3184281).
- [8] Y.-H. Chung et al., "SimPart: A simple yet effective replication-aided partitioning algorithm for logic simulation on GPU," in *Proc. Eur. Conf. Parallel Process.*, 2025, pp. 197–210.
- [9] Z. Guo, Y. Zhang, R. Wang, Y. Lin, and H. Ren, "GEM: GPU-accelerated emulator-inspired RTL simulation," in *Proc. 62nd ACM/IEEE Des. Autom. Conf. (DAC)*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 1–7, doi: [10.1109/DAC63849.2025.11132713](https://doi.org/10.1109/DAC63849.2025.11132713).
- [10] Y. Zhang, H. Ren, A. Sridharan, and B. Khailany, "GATSPi: GPU accelerated gate-level simulation for power improvement," in *Proc. 59th ACM/IEEE Des. Autom. Conf. (DAC)*, 2022, pp. 1231–1236.
- [11] Y.-H. Chung et al., "Accelerating gate sizing using GPU," in *Proc. Eur. Conf. Parallel Process.*, 2025, pp. 1–2.
- [12] G. Huang et al., "Machine learning for electronic design automation: A survey," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, no. 5, pp. 1–46, 2021, doi: [10.1145/3451179](https://doi.org/10.1145/3451179).
- [13] Y.-C. Lu, S. Pentapati, and S. K. Lim, "VLSI placement optimization using graph neural networks," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 6–12.
- [14] Y.-C. Lu, S. Pentapati, and S. K. Lim, "The law of attraction: Affinity-aware placement optimization using graph neural networks," in *Proc. Int. Symp. Physical Des. (ISPD)*, 2021, pp. 7–14.
- [15] Y.-C. Lu and S. K. Lim, "On advancing physical design using graph neural networks (invited paper)," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2022, pp. 1–7.
- [16] Y.-C. Lu, S. Nath, S. S. K. Pentapati, and S. K. Lim, "A fast learning-driven signoff power optimization framework," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2020, pp. 1–9.
- [17] Y.-C. Lu, S. Nath, S. S. K. Pentapati, and S. K. Lim, "ECO-GNN: Signoff power prediction using graph neural networks with subgraph approximation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 28, no. 4, pp. 1–22, 2023.
- [18] Y.-C. Lu, S. Nath, V. Khandelwal, and S. K. Lim, "Doomed run prediction in physical design by exploiting sequential flow and graph learning," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2021, pp. 1–9, doi: [10.1109/ICCAD51958.2021.9643435](https://doi.org/10.1109/ICCAD51958.2021.9643435).
- [19] Y.-C. Lu, H. Ren, H.-H. Hsiao, and S. K. Lim, "DREAM-GAN: Advancing DREAMPlace towards commercial-quality using generative adversarial learning," in *Proc. Int. Symp. Physical Des. (ISPD)*, 2023, pp. 141–148.
- [20] Y.-C. Lu, H. Ren, H.-H. Hsiao, and S. K. Lim, "GAN-Place: Advancing open source placers to commercial-quality using generative adversarial networks and transfer learning," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 2, pp. 1–17, 2024.
- [21] Y.-C. Lu, T. Yang, S. K. Lim, and H. Ren, "Placement optimization via PPA-directed graph clustering," in *Proc. ACM/IEEE 4th Workshop Mach. Learn. CAD (MLCAD)*, 2022, pp. 1–6, doi: [10.1145/3551901.3556482](https://doi.org/10.1145/3551901.3556482).
- [22] Y.-C. Lu, S. S. K. Pentapati, L. Zhu, K. Samadi, and S. K. Lim, "TP-GNN: A graph neural network framework for tier partitioning in monolithic 3D ICs," in *Proc. 7th ACM/IEEE Des. Automat. Conf. (DAC)*, 2020, pp. 1–6, doi: [10.1109/DAC18072.2020.9218582](https://doi.org/10.1109/DAC18072.2020.9218582).

- [23] P. Vanna-lampikul, C. Shao, Y.-C. Lu, S. Pentapati, and S. K. Lim, "Snap-3D: A constrained placement-driven physical design methodology for face-to-face-bonded 3D ICs," in *Proc. Int. Symp. Physical Des. (ISPD)*, 2021, pp. 39–46, doi: [10.1145/3439706.3447049](https://doi.org/10.1145/3439706.3447049).
- [24] H.-H. Hsiao, P. Vanna-lampikul, Y.-C. Lu, and S. K. Lim, "ML-based physical design parameter optimization for 3D ICs: From parameter selection to optimization," in *Proc. 61st ACM/IEEE Des. Automat. Conf. (DAC)*, 2024, pp. 1–6, doi: [10.1145/3649329.3656509](https://doi.org/10.1145/3649329.3656509).
- [25] H.-H. Hsiao et al., "DCO-3D: Differentiable congestion optimization in 3D ICs," in *Proc. 62nd ACM/IEEE Des. Automat. Conf. (DAC)*, 2025, pp. 1–7, doi: [10.1109/DAC63849.2025.11132448](https://doi.org/10.1109/DAC63849.2025.11132448).
- [26] Y.-C. Lu, S. Nath, V. Khandelwal, and S. K. Lim, "RL-Sizer: VLSI gate sizing for timing optimization using deep reinforcement learning," in *Proc. 58th ACM/IEEE Des. Automat. Conf. (DAC)*, 2021, pp. 733–738, doi: [10.1109/DAC18074.2021.9586138](https://doi.org/10.1109/DAC18074.2021.9586138).
- [27] S. Nath, G. Pradipta, C. Hu, T. Yang, B. Khailany, and H. Ren, "TransSizer: A novel transformer-based fast gate sizer," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2022, pp. 1–9.
- [28] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021, doi: [10.1038/s41586-021-03544-w](https://doi.org/10.1038/s41586-021-03544-w).
- [29] Y.-C. Lu, W.-T. Chan, D. Guo, S. Kundu, V. Khandelwal, and S. K. Lim, "RL-CCD: Concurrent clock and data optimization using attention-based self-supervised reinforcement learning," in *Proc. 60th ACM/IEEE Des. Automat. Conf. (DAC)*, 2023, pp. 1–6, doi: [10.1109/DAC56929.2023.10248008](https://doi.org/10.1109/DAC56929.2023.10248008).
- [30] H.-H. Hsiao, Y.-C. Lu, P. Vanna-lampikul, and S. K. Lim, "FastTuner: Transferable physical design parameter optimization using fast reinforcement learning," in *Proc. Int. Symp. Physical Des. (ISPD)*, 2024, pp. 93–101, doi: [10.1145/3626184.3633328](https://doi.org/10.1145/3626184.3633328).
- [31] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "RePlace: Advancing solution quality and routability validation in global placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 9, pp. 1717–1730, Sep. 2019, doi: [10.1109/TCAD.2018.2859220](https://doi.org/10.1109/TCAD.2018.2859220).
- [32] Z. Guo and Y. Lin, "Differentiable-timing-driven global placement," in *Proc. 59th ACM/IEEE Des. Automat. Conf. (DAC)*, 2022, pp. 1315–1320, doi: [10.1145/3489517.3530486](https://doi.org/10.1145/3489517.3530486).
- [33] J. A. Roy, N. Viswanathan, G. J. Nam, C. J. Alpert, and I. L. Markov, "CRISP: Congestion reduction by iterated spreading during placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. – Dig. Tech. Papers*, 2009, pp. 357–362, doi: [10.1145/1687399.1687467](https://doi.org/10.1145/1687399.1687467).
- [34] P. Spindler and F. M. Johannes, "Fast and accurate routing demand estimation for efficient routability-driven placement," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2007, pp. 1–6, doi: [10.1109/DATE.2007.364463](https://doi.org/10.1109/DATE.2007.364463).
- [35] R. Liang, A. Agnesina, G. Pradipta, V. A. Chhabria, and H. Ren, "Invited paper: CircuitOps: An ML infrastructure enabling generative AI for VLSI circuit optimization," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2023, pp. 1–6, doi: [10.1109/ICCAD57390.2023.10323611](https://doi.org/10.1109/ICCAD57390.2023.10323611).
- [36] T.-W. Huang and M. D. F. Wong, "OpenTimer: A high-performance timing analysis tool," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, 2015, pp. 895–902, doi: [10.1109/ICCAD.2015.7372666](https://doi.org/10.1109/ICCAD.2015.7372666).
- [37] C. Albrecht, "IWLS 2005 benchmarks," in *Proc. Int. Workshop Logic Synthesis*, 2005.
- [38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [39] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 6000–6010.
- [40] L. Rampášek et al., "Recipe for a general, powerful, scalable graph transformer," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 14,501–14,515.
- [41] Y.-C. Lu, K. Kunal, G. Pradipta, R. Liang, R. Gandikota, and H. Ren, "LEGO-size: LLM-enhanced GPU-optimized signoff-accurate differentiable VLSI gate sizing in advanced nodes," in *Proc. Int. Symp. Physical Des. (ISPD)*, 2025, pp. 152–162.
- [42] K. Choromanski et al., "Rethinking attention with performers," 2020, *arXiv:2009.14794*.
- [43] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [44] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [45] T.-B. Chan, A. B. Kahng, and J. Li, "OLO: A no-loop, predictive useful skew methodology for improved timing in IC implementation," in *Proc. 15th Int. Symp. Qual. Electron. Des.*, 2014, pp. 504–509, doi: [10.1109/ISQED.2014.6783368](https://doi.org/10.1109/ISQED.2014.6783368).
- [46] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1025–1035.
- [47] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [48] C. Chu and Y.-C. Wong, "FLUTE: Fast look-up table based rectilinear Steiner minimal tree algorithm for VLSI design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 1, pp. 70–83, Jan. 2008, doi: [10.1109/TCAD.2007.907068](https://doi.org/10.1109/TCAD.2007.907068).
- [49] J.-A. Désidéri, "Multiple-gradient descent algorithm for multiobjective optimization," *C.R. Math.*, vol. 350, no. 5–6, pp. 313–318, 2012.
- [50] S. Panth, K. Samadi, Y. Du, and S. K. Lim, "Shrunk-2-D: A physical design methodology to build commercial-quality monolithic 3-D ICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 36, no. 10, pp. 1716–1724, Oct. 2017, doi: [10.1109/TCAD.2017.2648839](https://doi.org/10.1109/TCAD.2017.2648839).
- [51] Y.-C. Lu, S. Pentapati, L. Zhu, G. Murali, K. Samadi, and S. K. Lim, "A machine learning-powered tier partitioning methodology for monolithic 3-D ICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4575–4586, Nov. 2022, doi: [10.1109/TCAD.2021.3139310](https://doi.org/10.1109/TCAD.2021.3139310).
- [52] Y.-H. Lu, H.-H. Hsiao, Y.-C. Lu, H. Ren, and S. K. Lim, "Differentiable tier assignment for timing and congestion-aware routing in 3d ICs," in *Proc. 31st Asia South Pacific Des. Automat. Conf. (ASP-DAC)*, 2026, pp. 475–481, doi: [10.1109/ASP-DAC66049.2026.11420682](https://doi.org/10.1109/ASP-DAC66049.2026.11420682).

About the Authors

Yi-Chen Lu (yclueecs@gmail.com) received his B.S. degree in electrical engineering from National Taiwan University, Taipei, Taiwan, and his Ph.D. degree in electrical and computer engineering from the Georgia Institute of

Technology, Atlanta, GA, USA, in 2023. He is a founding member of the technical staff at Rursive Intelligence, Santa Clara, CA 95051 USA. Previously, he was a senior research scientist at NVIDIA and a physical design methodology engineer at Apple. His research interests include machine learning for physical design, differentiable optimization, and GPU-accelerated design automation. He received best paper awards at DAC and ASP-DAC.

Hao-Hsiang Hsiao (thsiao@gatech.edu) is currently pursuing his Ph.D. degree at the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA. He is also a research intern at NVIDIA. His research interests include artificial intelligence/machine learning-assisted electronic design automation and 3D ICs.

Rongjian Liang (rliang@nvidia.com) received his Ph.D. degree from Texas A&M University, College Station, TX, USA. He is currently a senior research scientist in the Design Automation Research Group at NVIDIA, Santa Clara, CA 95051 USA. His research interests include machine learning for electronic design automation, hardware-software co-design, and machine learning (ML) infrastructure for VLSI optimization. He is the lead developer of CircuitOps, an open-source ML infrastructure for VLSI circuit optimization.

Haoxing Ren (markren@agentrys.ai) received his Ph.D. degree in computer engineering from the University of Texas at Austin, Austin, TX, USA. He is the founder and CEO of Agentrys, Santa Clara, CA 95051 USA. Previously, he was the director of design automation research at NVIDIA, and before that, he spent 15 years at IBM, where he led the development of placement tools, logic engineering change order synthesis, and high-level synthesis tools. He holds more than 30 patents and has coauthored more than 100 papers and books. He received the IBM Corporate Award and best paper awards at DAC, ASP-DAC, ISPD, IEEE TCAD, MLCAD, and LAD. He is a Fellow of IEEE. 